Can regularization priors improve dataset distillation ?

Jibril El Hassani

February 2026

1 Introduction

[1] Introduces dataset distillation, a novel method to learn the best images to train a model on as little data as possible. This paper shows impressive results, as the authors were able to obtain near-perfect accuracy on MNIST (94%) by training the model on only 10 images (one per class) ! Those images are not selected from the dataset, rather, the authors learn the images through an optimization process. The goal of this report is first to reproduce some of the results in [1], and try to improve the authors' work by using optimization principles learned in the OVO course.

2 Motivations

When it comes to text and images, major AI companies have scraped most of what is available on the internet. While data quantity has proven to be a great way to improve model performance, research is now shifting towards data quality, as data scraping is facing a bottleneck. Dataset efficiency, defined as obtaining the best generalization performance with as few data samples as possible, has been a very important topic in recent deep learning research. To achieve this goal, multiple approaches were proposed : [2] uses hierarchical k-means to obtain canonical images for each concept whereas [3] uses bootstrap sampling of a larger dataset to minimize time to accuracy. The goal of these methods is often to use the smallest curated dataset possible to minimize training cost. The authors of dataset distillation introduce an original concept where, instead of choosing the best possible images in the dataset, synthtetic images are learned to be optimal for the downstream task. This relaxes the constraint that the images have to be real images from a dataset and gives more freedom on which features can be present in the curated train set, hence the extreme data efficiency of the method. The main contribution of the author is the introduction of this new distillation strategy as well as numerous benchmarks to prove its efficiency. We only reproduce the results on MNIST (and not CIFAR-10) and

only compare to random sampling (and not k-means and image averaging) in this paper.

3 Problem statement

We want to frame the learning of images as an optimization problem. Let I be a fixed set of images, we will be trying to have :

$$I = \text{argmin}_{x \in \mathbf{R}^p} f(x)$$

Where a minimum of f should correspond to images that, when trained on, give the best generalization accuracy. The trick lies in designing a function f that satisfies this property.

The authors of [1] build on the following idea : the best images are the ones that help reduce the loss the most when an untrained model performs a gradient descent step on them. Mathematically, let θ_0 be an randomly initialized set of parameters, the first gradient descent step on our set of images reads :

$$\theta_1 = \theta_0 - \eta \nabla J(\theta_0, I)$$

Where J is the loss function (cross entropy in the image classification case) and I is our set of images. Now we can introduce the function we want to minimize : we take a single gradient descent step on our set of images, then evaluate θ_1. The loss of θ_1 will be used to build the images :

$$f(I) = J(\theta_0 - \eta \nabla J(\theta_0, I), B)$$

Where we introduced B, a batch of data from the dataset we are trying to distill. We can minimize this function using gradient descent. To get the Algorithm 1 introduced in [1] and available in Appendix A, we only need two more tricks :

- We do not use a single θ_0, but rather, we sample multiple randomly initialized models at each iteration to avoid the high variance of a single sample monte Carlo estimation.
- The learning rate used for the single GD step (η) is learned during optimization along the images.

4 Reproduction plan

As I found the results of this paper really interesting, I first started by reproducing the results on MNIST. After noticing that this distillation method does not scale very well with data, I tried to improve performance by adding regularization priors seen in the course (which has not been explored in the paper). I tested different regularizations :

$$\text{Smoothness prior : } \lambda||I||_2^2$$

$$\text{Sparsity prior : } \lambda||I||_1$$

$$\text{Learnable prior : } \lambda \text{ Crossentropy(model(I), labels)}$$

The third one penalizes images for which a pretrained model is not accurate enough. More ablations on priors can be found in the appendices B and C.

5 Results

The results are summarized in Figure 1. We see all the strength of dataset distillation in the 10 samples regime : training to convergence on 10 images takes a few seconds but yields a relatively good performance when using dataset distillation. To my disappointment, the regularization prior approach did not work for any of the priors tested. I also noticed that this method does not work very well if we try to scale up the number of distilled images as the images get very noisy (see Appendix C for more details) and the optimization starts requiring a lot of compute (because more models need to be sampled at each iteration to make the images less noisy). I also noticed that the learned model prior (Fig 2d)) does not seem to push the images to look 'hand-written' which is what I hoped. My hypothesis is that the learned priors mentioned in the OVO course are foundation models, trained on a lot of natural images whereas this prior was just made using a two layer MLP pretrained on MNIST.

References

[1] WANG, Tongzhou, ZHU, Jun-Yan, TORRALBA, Antonio, et al. Dataset distillation. arXiv preprint arXiv:1811.10959, 2018.

[2] VO, Huy V., KHALIDOV, Vasil, DARCET, Timothée, et al. Automatic data curation for self-supervised learning: A clustering-based approach. arXiv preprint arXiv:2405.15613, 2024.

[3] OKANOVIC, Patrik, WALEFFE, Roger, MAGEIRAKOS, Vasilis, et al. Repeated random sampling for minimizing the time-to-accuracy of learning. arXiv preprint arXiv:2305.18424, 2023.

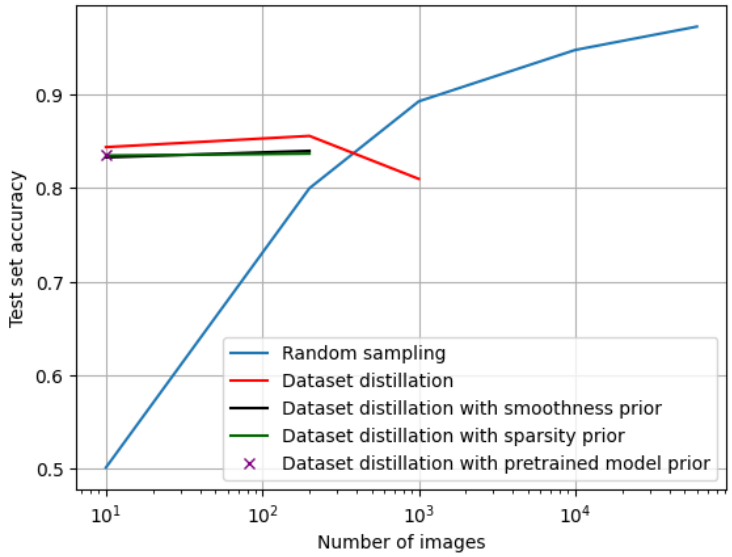
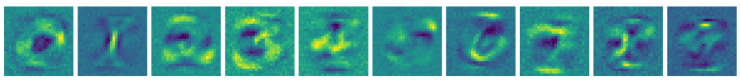
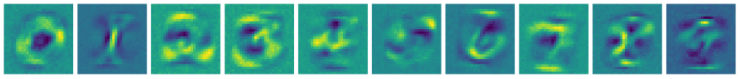


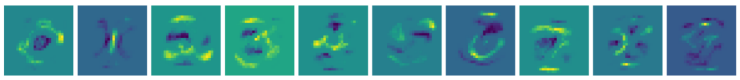
Figure 1: Performance of the different distillation methods with respect to the amount of data (log scale). We see that the priors do not offer any significant performance boost and that dataset distillation is unable to scale well in our use case. Those two aspects were not explored in the paper.



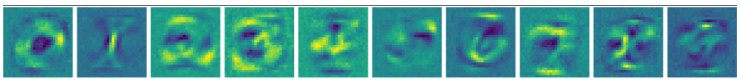
(a)) Distilled images with vanilla image distillation : we can see the discriminative features for each number as well as the sharp contrast.



(b)) Distilled images with smoothness prior



(c)) Distilled images with sparsity prior, the effect of the regularization is easier to see on this one !



(d)) Distilled images with learned model prior

Figure 2: Distilled images with different priors.

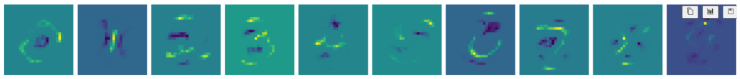
Appendix A Dataset distillation Algorithm

Algorithm 1 Dataset Distillation

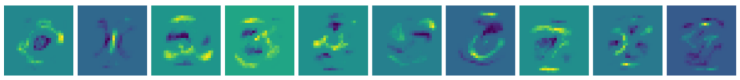
Require: $p(\theta_0)$: distribution of initial weights; M: the number of distilled data
Require: α: step size; n: batch size; T: the number of optimization iterations; $\tilde{\eta}_0$: initial value for $\tilde{\eta}$

Initialize $\tilde{\mathbf{x}} = \{\tilde{x}_i\}_{i=1}^{M}$ randomly, $\tilde{\eta} \leftarrow \tilde{\eta}_0$
for each training step $t = 1$ to T **do**
    Get a minibatch of real training data $\mathbf{x}_t = \{x_{t,j}\}_{j=1}^{n}$
    Sample a batch of initial weights $\theta_0^{(j)} \sim p(\theta_0)$
    for each sampled $\theta_0^{(j)}$ **do**
        Compute updated parameter with GD: $\theta^{(j)} = \theta_0^{(j)} - \tilde{\eta}\nabla_{\theta_0^{(j)}}\ell(\tilde{\mathbf{x}}, \theta_0^{(j)})$
        Evaluate the objective function on real training data: $\mathcal{L}^{(j)} = \ell(\mathbf{x}_t, \theta^{(j)})$
    end for
    Update $\tilde{\mathbf{x}} \leftarrow \tilde{\mathbf{x}} - \alpha\nabla_{\tilde{\mathbf{x}}}\sum_j \mathcal{L}^{(j)}$, and $\tilde{\eta} \leftarrow \tilde{\eta} - \alpha\nabla_{\tilde{\eta}}\sum_j \mathcal{L}^{(j)}$
end for
return distilled data $\tilde{\mathbf{x}}$ and optimized learning rate $\tilde{\eta}$

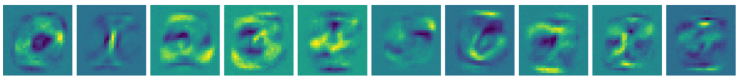
Appendix B Influence of the regularization hyperparameter on the distilled images



(a)) Distilled images with sparsity prior, lambda = 1e-2



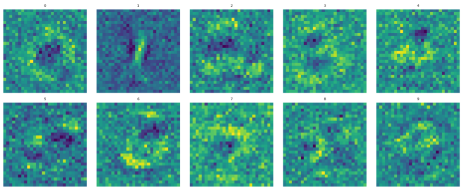
(b)) Distilled images with sparsity prior, lambda = 1e-3



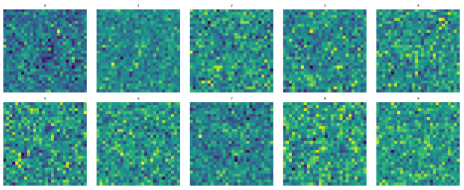
(c)) Distilled images with sparsity prior, lambda = 1e-4

Figure 3: Change in the image when changing the lambda hypeparameter in the sparsity regularization term. We clearly see the images getting pruned when the lambda is highest (fig 3a)). While this is very interesting to analyze which hand written features are the most discriminative, data shows that performance is best when lambda is minimal (fig 3c))

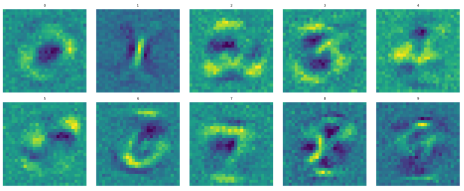
Appendix C When using more distilled images, each image individually get worse



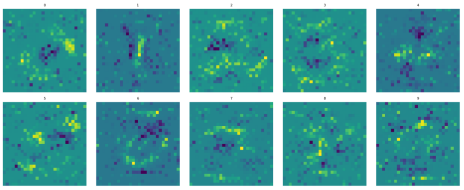
(a)) First 10 distilled images when creating 200 distilled images



(b)) First 10 distilled images when creating 1000 distilled images



(c)) First 10 distilled images when creating 200 distilled images with smoothness prior



(d)) First 10 distilled images when creating 200 distilled images with sparsity prior

Figure 4: When scaling the number of images, each image individually gets more noisy (see fig 4a) and 4b)). We can try to adress this by adding a smoothness prior (fig 4c)) or a sparsity prior (fig 4d)) but data shows that this does not improve substantially the performance.